



Module 4: The Heart of Rust

Lesson 4.1: Ownership

Ownership is Rust's zero-cost abstraction for memory safety, defined by three main rules: every value has a single owner, and when that owner goes out of scope, the value is immediately dropped (its memory is freed). When a complex value with heap data (like String) is assigned to a new variable, the value is moved, invalidating the original owner to prevent double-free errors.

Key Takeaways

- The "move" operation copies stack data (pointer, length, capacity) but invalidates the old owner, ensuring only one variable is responsible for cleaning up the heap data.
- If a true duplicate of the data (both stack and heap) is desired, the expensive `.clone()` method must be explicitly called.
- Passing ownership of a value to a function means the function consumes the value, and the original variable cannot be used afterwards.
- Rust distinguishes between copy (only stack data copied) and clone (stack and heap data copied).

Key Concepts

- **Ownership Rules:** 1. Each value has an owner. 2. Only one owner exists. 3. Value is dropped when the owner goes out of scope.
- **Move:** The transfer of ownership that occurs during assignment for heap-backed data types, invalidating the previous variable.
- **Clone (Deep Copy):** An explicit operation that creates a full, independent copy of both stack and heap data.
- **Stack vs. Heap:** The stack is fast and predictable (fixed-size values); the heap is slower (variable-size, dynamically allocated values).



Module 4: The Heart of Rust

Lesson 4.2: References & Borrowing

Borrowing uses references (e.g., `&String`) to access a value without taking ownership, allowing the original owner to retain the value. References are immutable by default, but a mutable reference (`&mut`) can be created for a mutable value, allowing the reference to modify the data.

Key Takeaways

- The compiler enforces a crucial safety rule: a value can have either one mutable reference (`&mut`) OR any number of immutable references (`&`) at any given time.
- References are protected by lifetimes, a concept guaranteeing that a reference never outlives the data it points to.
- The dot operator (`.`) simplifies code by automatically dereferencing references when calling methods or accessing fields.
- For all other operations (like assignment or arithmetic), manual dereferencing using the asterisk (`*`) is required.

Key Concepts

- **Borrowing/Reference (`&`):** Temporarily accessing a value; the reference itself is moved/copied, not the underlying data.
- **Mutable Reference (`&mut`):** An "exclusive" reference that allows changing the value it points to.
- **Dereferencing (`*`):** Manually accessing the value behind a reference; needed for operations where the dot operator's auto-dereferencing doesn't apply.
- **Borrowing Rule:** The compiler-enforced invariant that prevents data races by limiting mutable access (one `&mut`) when shared immutable access (many `&`) is also present.



Module 4: The Heart of Rust

Lesson 4.3: Exercise G - Ownership & References

Exercise G required implementing functions that correctly manage ownership: `inspect` took an immutable reference (`&String`) for reading, `change` took a mutable reference (`&mut String`) for modification, and `eat` accepted the owned `String` to consume the value. The exercise also demonstrated using an `if` expression as a tail expression to return a boolean result concisely.

Key Takeaways

- Using an immutable reference (`&`) is the correct pattern when a function only needs to read the data and avoid taking ownership.
- If a function needs to change the content of a string, it must receive a mutable reference (`&mut`) to a mutable variable.
- Accepting the actual value (`String`) moves ownership into the function, meaning the caller cannot use the variable again.
- Conditional logic can be simplified by returning the result of the `if` expression directly, removing explicit `return true` or `return false` statements.

Key Concepts

- **Immutable Borrowing:** The safest method for passing data to functions that only require read access.
- **Consuming Ownership:** Accepting ownership of a type (e.g., `String`) into a function, implying the value will be used up or dropped.
- **Idiomatic Boolean Return:** Returning the result of a conditional expression directly as the function's tail expression.